



第四節 使用 CSS 美化網頁

HTML網頁設計課程

講師：巫宏鈞

課程單元

- CSS 樣式規則
- CSS 選擇器
- CSS 樣式
- CSS 盒模型
- CSS 定位

CSS 樣式規則

- CSS 規則是由兩個主要的部分構成：
 - 選擇器
 - 一條或多條樣式聲明

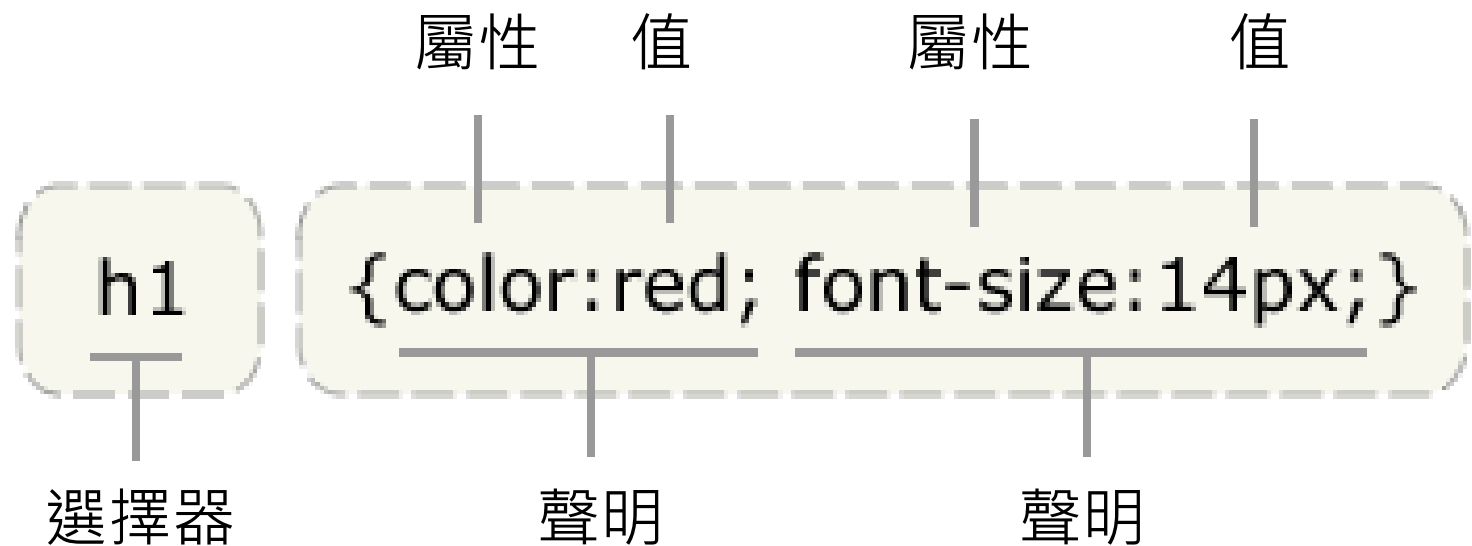
- 語法如下

```
selector {  
    property1: value;  
    property2: value;  
}
```

CSS 樣式規則

- 下面是設定標題一的字體為紅色，大小為14個像素

```
h1 { color:red; font-size:14px; }
```



CSS 樣式規則

- CSS 套用方式
 - 行內套用 (Inline)
 - 嵌入套用 (Embed)
 - 外部連接套用 (External Link)
 - 匯入套用 (Import)

CSS 樣式規則

- 行內套用

- 我們可以在 HTML 文件內直接宣告樣式。

```
<p style='font-family:verdana; font-size:16;'>
```

This is font size 16.

```
</p>
```

CSS 樣式規則

- 嵌入套用

- 樣式可以嵌入於 HTML 文件中 (通常是在 `<head>` 內)。

```
<head>  
  <style type="text/css">  
    p {  
      background-color:#FF0000;  
    }  
  </style>  
</head>
```

CSS 樣式規則

- 外部連接套用

- 在這種方式下，所有的 CSS 樣式宣告都是存在另外一個檔案中。該檔案通常名稱為 .css。在 HTML 文件的 `<head> .. </head>` 之中，我們將用以下的程式碼將這個 .css 檔案連接進入：

```
<link rel=stylesheet type="text/css"  
      href="external-stylesheet.css">
```


CSS 樣式規則

- 匯入套用

- 外部的 CSS 樣式也可以被匯入進 HTML 文件。

```
<STYLE TYPE="text/css">  
<!--  
    @import  
    url(http://www.mysite.com/style.css);  
-->  
</STYLE>
```

CSS 樣式規則

- 媒體類別

- 不同的媒體類別可以用不同的 CSS 樣式表。

- 外部連接套用

- ```
<link rel="stylesheet" href="style.css" type="text/css" media="媒體類別">
```

- 嵌入套用

- ```
<style type="text/css" media="媒體類別">
```

- 匯入套用

- ```
@import url("style.css") 媒體類別;
```

# CSS 樣式規則

- 媒體類別可以有以下的值：

| 值          | 適用於                    |
|------------|------------------------|
| all        | 所有媒體類別                 |
| aural      | 言語合成器                  |
| braille    | 布拉耶點字法 (盲人用) 媒體        |
| handheld   | 掌上型媒體                  |
| print      | 列印                     |
| projection | 投射媒體                   |
| screen     | 電腦螢幕                   |
| tty        | 固定字寬 (fixed-pitch) 的媒體 |
| tv         | 電視類的媒體                 |

# CSS 樣式規則

- 串接順序

- 串接是指當不同樣式表中對相同屬性有不同定義時，應該要用哪一個樣式表中的定義的規則。
- 從最高優先權到最低優先權的排名如下：
  - 內行套用的樣式表 (Inline stylesheet)
  - 嵌入套用的樣式表 (Embedded stylesheet)
  - 匯入套用的樣式表 (Imported stylesheet)
  - 外部連接套用的樣式表 (Linked stylesheet)
  - 瀏覽器本身的樣式表 (Browser's own stylesheet)

# CSS 選擇器

- 群組選擇器

- 如果有數個選擇器享有同樣的樣式，它們可以同時被宣告。這叫做「grouping」。舉例來說，如果 `<h1>`, `<h2>`, and `<h3>` 都會有相同的樣式，那它們就可以用以下的方式被宣告：被分組的選擇器可以分享相同的聲明。用逗號將需要分組的選擇器分開。

```
h1, h2, h3 {
 color: green;
}
```

# CSS 選擇器

- 後代選擇器

- 後代選擇器允許你根據文檔的上下文關係來確定某個標籤的樣式。通過合理地使用，我們可以使HTML代碼變得更加整潔。

```
li strong {
 font-style: italic;
 font-weight: normal;
}
```

- 請注意標記為 `<strong>` 的藍色代碼的上下文關係：

```
<p> 我是粗體字，不是斜體字 </p>
```

```

 我是斜體字
 我是正常的字體

```

# CSS 選擇器

- Class 選擇器

- 在 CSS 中，類別選擇器以一個點號顯示：

```
.center { text-align: center }
```

- 在下面的HTML代碼中，h1 和 p 元素都有 center 類。這意味著兩者都將遵守 .center 選擇器中的規則。

```
<h1 class="center">
 This heading will be center-aligned
</h1>
```

```
<p class="center">
 This paragraph will also be center-aligned.
</p>
```

# CSS 選擇器

- ID 選擇器

- 在 CSS 中，ID 選擇器以一個 # 號顯示：

```
#red{ color: red; }
#green{ color: green; }
```

- 下面的HTML 代碼中，id 屬性為red 的p 元素顯示為紅色，而id 屬性為green 的p 元素顯示為綠色。

```
<p id="red">這個段落是紅色</p>
<p id="green">這個段落是綠色</p>
```



# CSS 選擇器

- 屬性和值選擇器
  - 可以為擁有指定屬性的HTML 元素設置樣式，而不僅限於class 和id 屬性。
  - 下面的例子為帶有title 屬性的所有元素設置樣式：

```
[title] { color: red }
```

- 下面的例子為 title="Hello" 的所有元素設置樣式：

```
[title=Hello] { border:5px solid blue }
```

# CSS 樣式

- 背景色彩與圖片

- 可以使用 `background-color` 屬性為元素設置背景色。下面這條規則把元素的背景設置為灰色：

```
p { background-color: gray }
```

- 也可以使用 `background-image` 屬性把圖像放入背景。如果需要設置一個背景圖像，必須為這個屬性設置一個URL 值。下面例子為一個段落設置了一個背景：

```
p { background-image: url(images/bg.gif) }
```

# 操作範例 web08

- 請完成下面頁面設計：



# CSS 樣式

- 背景重複

- 如果需要在頁面上對背景圖像進行平鋪，可以使用 `background-repeat` 屬性。屬性值 `repeat-x` 和 `repeat-y` 分別導致圖像只在水平或垂直方向上重複，`no-repeat` 則不允許圖像在任何方向上平鋪。

```
body {
 background-image: url(images/03.gif);
 background-repeat: repeat-y;
}
```

# CSS 樣式

- 背景定位

- 可以利用 background-position 屬性改變圖像在背景中的位置 (先水平再垂直)。
- 下面的例子在 body 元素中將一個背景圖像居中放置：。

```
body {
 background-image: url(images/03.gif);
 background-repeat: no-repeat;
 background-position: center center;
}
```

# CSS 樣式

- 文字縮排與凸排
  - 通過使用 `text-indent` 屬性，所有元素的第一行都可以縮進一個指定的長度，甚至該長度可以是負值。下面的規則會使所有段落的首行縮排 5 em：

```
p { text-indent: 5em }
```

- 下面的規則會使所有段落的首行凸排 5 em：

```
p {
 text-indent: -5em;
 padding-left: 5em;
}
```

# CSS 樣式

- 水平對齊
  - text-align 是一個基本的屬性，它會影響一個元素中的文本行互相之間的對齊方式。
  - text-align 的值：
    - left - 靠左對齊
    - right - 靠右對齊
    - center - 置中對齊
    - justify - 左右對齊
  - 下面的規則會使所有段落置中對齊：

```
p { text-align: center }
```

# CSS 樣式

- 文字間隔與字母間隔
  - word-spacing 屬性可以改變字（單詞）之間的標準間隔。下面的規則會使所有段落中的文字間隔 30 像素：

```
p { word-spacing: 30px }
```

- letter-spacing 屬性可以修改字符或字母之間的間隔。下面的規則會使所有段落中的文字間隔 5 像素：

```
p { letter-spacing: 5px }
```



# CSS 樣式

- 文字裝飾
  - text-decoration 屬性，提供了很多非常有趣的行為。
  - text-decoration 的值：
    - none - 關閉裝飾
    - underline - 加底端線
    - overline - 加頂端線
    - line-through - 加刪除線
    - blink - 文字閃礫
  - 下面樣式可以去掉超鏈接的下劃線

`a { text-decoration: none }`

# CSS 樣式

- 字體
  - 使用 `font-family` 屬性定義文本的字體系列。  
下面為設定標題為「標楷體」

```
h1 { font-family: 標楷體 }
```

- 字體風格
  - `font-style` 屬性最常用於規定斜體文本。
  - 該屬性有三個值：
    - `normal` - 文本正常顯示
    - `italic` - 文本斜體顯示
    - `oblique` - 文本傾斜顯示

```
p { font-style: italic }
```

# CSS 樣式

- 字體加粗

- font-weight 屬性設置文本的粗細。使用bold 關鍵字可以將文本設置為粗體
- 關鍵字100 ~ 900 為字體指定了9 級加粗度。100 對應最細的字體變形，900 對應最粗的字體變形。數字400 等價於normal，而700 等價於bold。

```
p { font-weight: bold }
```

- 字體大小

- font-size 屬性設置文本的大小。  
font-size 值可以是絕對或相對值。文本大小預設是 16 像素。因此 1em 的默認尺寸是 16 像素。
- 可以使用下面這個公式將像素轉換為 em： $\text{pixels} / 16 = \text{em}$

```
h1 { font-size: 60px }
p { font-size: 0.875em } /* 14px/16=0.875em */
```

# CSS 樣式

- 超連結樣式

- 超連結的特殊性在於能夠根據它們所處的狀態來設置它們的樣式。

- 鏈接的四種狀態：

- a:link - 普通的、未被訪問的鏈接
    - a:visited - 用戶已訪問的鏈接
    - a:hover - 鼠標指針位於鏈接的上方
    - a:active - 鏈接被點擊的時刻

a:link { color: #FF0000 }

a:visited { color: #00FF00 }

a:hover { color: #FF00FF }  
上\*/

a:active { color: #0000FF }

/\*未被訪問的鏈接\*/

/\*已被訪問的鏈接\*/

/\*鼠標指針移動到鏈接

/\*正在被點擊的鏈接\*/

# CSS 樣式

- 列表樣式

- 要修改用於列表項的標誌類型，可以使用屬性 `list-style-type`：

| 值                        | 描述                            |
|--------------------------|-------------------------------|
| <code>none</code>        | 無標記。                          |
| <code>disc</code>        | 默認。標記是實心圓。                    |
| <code>circle</code>      | 標記是空心圓。                       |
| <code>square</code>      | 標記是實心方塊。                      |
| <code>decimal</code>     | 標記是數字。                        |
| <code>lower-roman</code> | 小寫羅馬數字(i, ii, iii, iv, v, 等。) |
| <code>upper-roman</code> | 大寫羅馬數字(I, II, III, IV, V, 等。) |
| <code>lower-alpha</code> | 小寫英文字母 (a, b, c, d, e, 等。)    |
| <code>upper-alpha</code> | 大寫英文字母 (A, B, C, D, E, 等。)    |
| <code>lower-latin</code> | 小寫拉丁字母(a, b, c, d, e, 等。)     |
| <code>upper-latin</code> | 大寫拉丁字母(A, B, C, D, E, 等。)     |

# CSS 樣式

- 列表圖像

- 只需要簡單地使用一個url() 值，就可以使用圖像作為標誌。

`ul { list-style-image : url(xxx.gif) }`

- 列表標誌位置

- 可以利用 list-style-position 確定標誌出現在列表項內容之外還是內容內部。

`ul { list-style-position: inside }`

# CSS 樣式

- 表格邊框

- 使用 border 屬性設置表格邊框。  
border 順序為：寬 線型 產色

```
table, th, td { border: 1px solid blue }
```

- 折疊邊框

- border-collapse 屬性設置是否將表格邊框折疊為單一邊框：

```
table { border-collapse: collapse }
```

# CSS 樣式

- 框線屬性

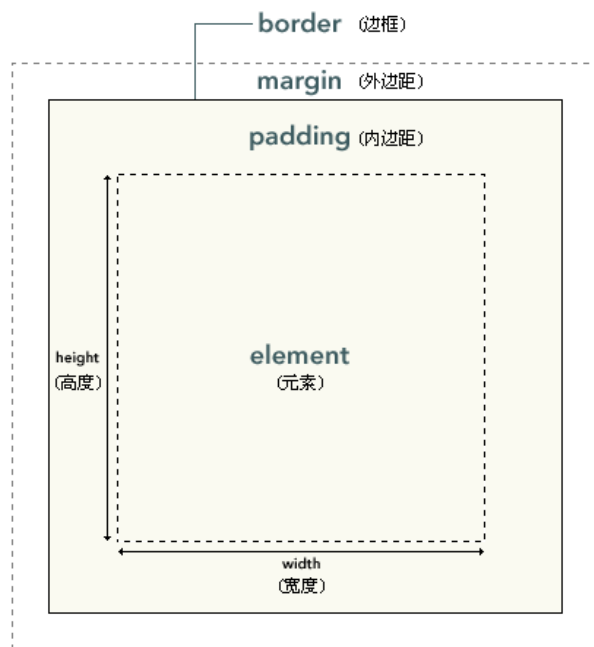
| 值      | 描述                                    |
|--------|---------------------------------------|
| none   | 定義無邊框。                                |
| dotted | 定義點狀邊框。在大多數瀏覽器中呈現為實線。                 |
| dashed | 定義虛線。在大多數瀏覽器中呈現為實線。                   |
| solid  | 定義實線。                                 |
| double | 定義雙線。雙線的寬度等於border-width 的值。          |
| groove | 定義3D 凹槽邊框。其效果取決於border-color 的值。      |
| ridge  | 定義3D 壟狀邊框。其效果取決於border-color 的值。      |
| inset  | 定義3D inset 邊框。其效果取決於border-color 的值。  |
| outset | 定義3D outset 邊框。其效果取決於border-color 的值。 |



# CSS 盒模型

- 盒模型概述

- 盒模型 (box model) 是用來描述一個元素是如何組成的。以下是盒子模式的式樣：



# CSS 盒模型

- 邊框的樣式
  - border-style 屬性定義了10 個不同的非 inherit 樣式，包括 none。

| 值       | 描述                                        |
|---------|-------------------------------------------|
| none    | 定義無邊框。                                    |
| hidden  | 與“none”相同。不過應用於表格時除外，對於表，hidden 用於解決邊框衝突。 |
| dotted  | 定義點狀邊框。在大多數瀏覽器中呈現為實線。                     |
| dashed  | 定義虛線。在大多數瀏覽器中呈現為實線。                       |
| solid   | 定義實線。                                     |
| double  | 定義雙線。雙線的寬度等於border-width 的值。              |
| groove  | 定義3D 凹槽邊框。其效果取決於border-color 的值。          |
| ridge   | 定義3D 壟狀邊框。其效果取決於border-color 的值。          |
| inset   | 定義3D inset 邊框。其效果取決於border-color 的值。      |
| outset  | 定義3D outset 邊框。其效果取決於border-color 的值。     |
| inherit | 規定應該從父元素繼承邊框樣式。                           |

# CSS 盒模型

- 定義單邊邊框樣式

- 您可以為一個邊框定義多個樣式，例如：

```
p { border-style: solid dotted dashed double }
```

- 如果希望為元素框的某一個邊設置邊框樣式，而不是設置所有 4 個邊的邊框樣式，可以使用下面的單邊邊框樣式屬性：
  - border-top-style - 上邊框樣式
  - border-right-style - 右邊框樣式
  - border-bottom-style - 下邊框樣式
  - border-left-style - 左邊框樣式

# CSS 盒模型

- 外邊距
  - 設置外邊距的最簡單的方法就是使用 `margin` 屬性。`margin` 屬性接受任何長度單位，可以是像素、英寸、毫米或`em`。
  - `margin` 可以設置為`auto`。
  - 下面的例子為`h1` 元素的四個邊分別定義了不同的外邊距，所使用的長度單位是像素：

```
h1 { margin : 10px 0px 15px 5px }
```

- 您可以使用下列任何一個屬性來只設置相應上的外邊距，而不會直接影響所有其他外邊距：
  - `margin-top` - 上邊距
  - `margin-right` - 右邊距
  - `margin-bottom` - 下邊距
  - `margin-left` - 左邊距

# CSS 盒模型

- 尺寸
  - CSS 尺寸屬性允許你控制元素的高度和寬度。

| 屬性                                 | 描述         |
|------------------------------------|------------|
| <a href="#"><u>height</u></a>      | 設置元素的高度。   |
| <a href="#"><u>line-height</u></a> | 設置行高。      |
| <a href="#"><u>max-height</u></a>  | 設置元素的最大高度。 |
| <a href="#"><u>max-width</u></a>   | 設置元素的最大寬度。 |
| <a href="#"><u>min-height</u></a>  | 設置元素的最小高度。 |
| <a href="#"><u>min-width</u></a>   | 設置元素的最小寬度。 |
| <a href="#"><u>width</u></a>       | 設置元素的寬度。   |

# CSS 盒模型

- 顯示與隱藏

- `visibility` 屬性規定元素是否可見。即使不可見的元素也會佔據頁面上的空間。

```
div { visibility: hidden /* 隱藏 */ }
div { visibility: visible /* 顯示 */ }
```

- `display` 屬性則可用來創建不佔據頁面空間的不可見元素。

```
div { display: none }
```

# CSS 定位

- CSS 定位機制
  - CSS 有三種基本的定位機制：普通流、浮動和絕對定位。
  - 除非專門指定，否則所有框都在普通流中定位。也就是說，普通流中的元素的位置由元素在 HTML 中的位置決定。
  - 塊級框從上到下一個接一個地排列，框之間的垂直距離是由框的垂直外邊距計算出來。

# CSS 定位

- CSS 定位屬性

| 屬性                                    | 描述                             |
|---------------------------------------|--------------------------------|
| <a href="#"><u>position</u></a>       | 把元素放置到一個靜態的、相對的、絕對的、或固定的位置中。   |
| <a href="#"><u>top</u></a>            | 定義了一個定位元素的上外邊距邊界與其包含塊上邊界之間的偏移。 |
| <a href="#"><u>right</u></a>          | 定義了定位元素右外邊距邊界與其包含塊右邊界之間的偏移。    |
| <a href="#"><u>bottom</u></a>         | 定義了定位元素下外邊距邊界與其包含塊下邊界之間的偏移。    |
| <a href="#"><u>left</u></a>           | 定義了定位元素左外邊距邊界與其包含塊左邊界之間的偏移。    |
| <a href="#"><u>overflow</u></a>       | 設置當元素的內容溢出其區域時發生的事情。           |
| <a href="#"><u>clip</u></a>           | 設置元素的形狀。元素被剪入這個形狀之中，然後顯示出來。    |
| <a href="#"><u>vertical-align</u></a> | 設置元素的垂直對齊方式。                   |
| <a href="#"><u>z-index</u></a>        | 設置元素的堆疊順序。                     |



# CSS 定位

- 通過使用 `position` 屬性，我們可以選擇4種不同類型的定位，這會影響元素框生成的方式。
- `position` 屬性值的含義：
  - `static`  
元素框正常生成。塊級元素生成一個矩形框，作為文檔流的一部分，行內元素則會創建一個或多個行框，置於其父元素中。
  - `relative`  
元素框偏移某個距離。元素仍保持其未定位前的形狀，它原本所佔的空間仍保留。
  - `absolute`  
元素框從文檔流完全刪除，並相對於其包含塊定位。包含塊可能是文檔中的另一個元素或者是初始包含塊。元素原先在正常文檔流中所佔的空間會關閉，就好像元素原來不存在一樣。元素定位後生成一個塊級框，而不論原來它在正常流中生成何種類型的框。
  - `fixed`  
元素框的表現類似於將 `position` 設置為 `absolute`，不過其包含塊是視窗本身。

# CSS 定位

- `overflow` 屬性規定當內容溢出元素框時發生的事情。

```
div { width: 150px; height: 150px; overflow: scroll; }
```

| 值                    | 描述                                     |
|----------------------|----------------------------------------|
| <code>visible</code> | 默認值。內容不會被修剪，會呈現在元素框之外。                 |
| <code>hidden</code>  | 內容會被修剪，並且其餘內容是不可見的。                    |
| <code>scroll</code>  | 內容會被修剪，但是瀏覽器會顯示滾動條以便查看其餘的內容。           |
| <code>auto</code>    | 如果內容被修剪，則瀏覽器會顯示滾動條以便查看其餘的內容。           |
| <code>inherit</code> | 規定應該從父元素繼承 <code>overflow</code> 屬性的值。 |

# CSS 定位

- clip 屬性剪裁絕對定位元素。
- 當一幅圖像的尺寸大於包含它的元素時設定 clip 可以規定一個元素的可見尺寸，這樣此元素就會被修剪並顯示為這個形狀。

```
img {
 position: absolute;
 clip: rect(0px, 60px, 200px, 0px);
}
```

# CSS 定位

- vertical-align 屬性設置元素的垂直對齊方式。

`img { vertical-align: text-top }`

| 值           | 描述                  |
|-------------|---------------------|
| top         | 把元素的頂端與行中最高元素的頂端對齊  |
| text-top    | 把元素的頂端與父元素字體的頂端對齊   |
| middle      | 把此元素放置在父元素的中部。      |
| bottom      | 把元素的頂端與行中最低的元素頂端對齊。 |
| text-bottom | 把元素的底端與父元素字體的底端對齊。  |

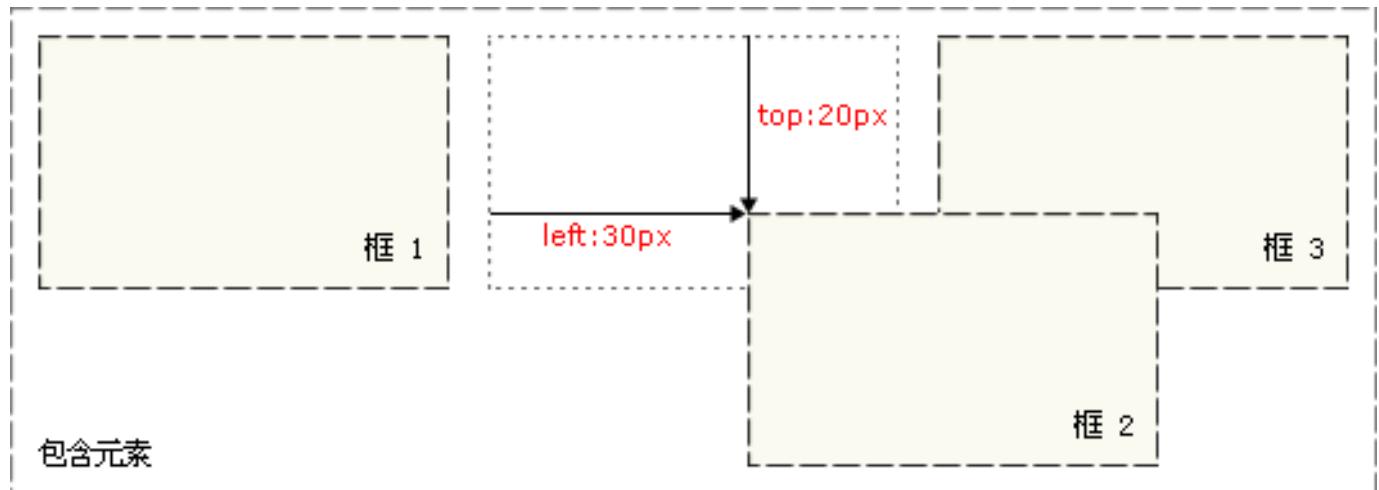
# CSS 定位

- z-index 屬性設置元素的堆疊順序。擁有更高堆疊順序的元素總是會處於堆疊順序較低的元素的前面。
- 注意：z-index 僅能在定位元素上奏效

```
img {
 position: absolute;
 left: 0px;
 top: 0px;
 z-index: -1;
}
```

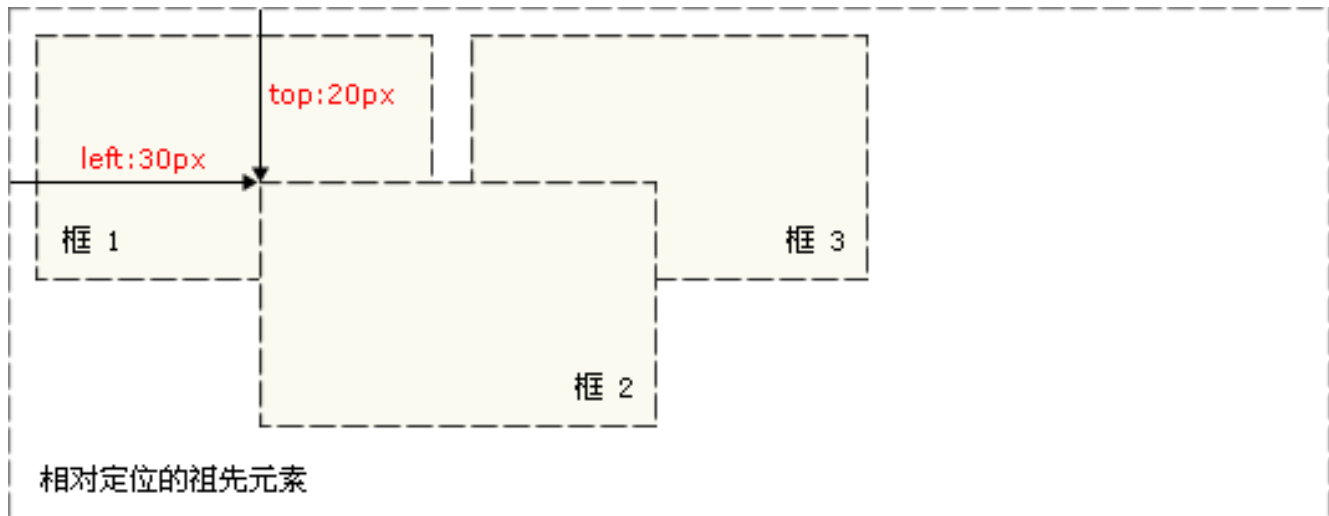
# CSS 定位

- CSS 相對定位
  - `div {`  
    `position: relative;`  
    `left: 30px;`  
    `top: 20px;`  
    `}`



# CSS 定位

- CSS 絕對定位
  - `div {`  
    `position: absolute;`  
    `left: 30px;`  
    `top: 20px;`  
    `}`



# CSS 定位

- CSS 浮動

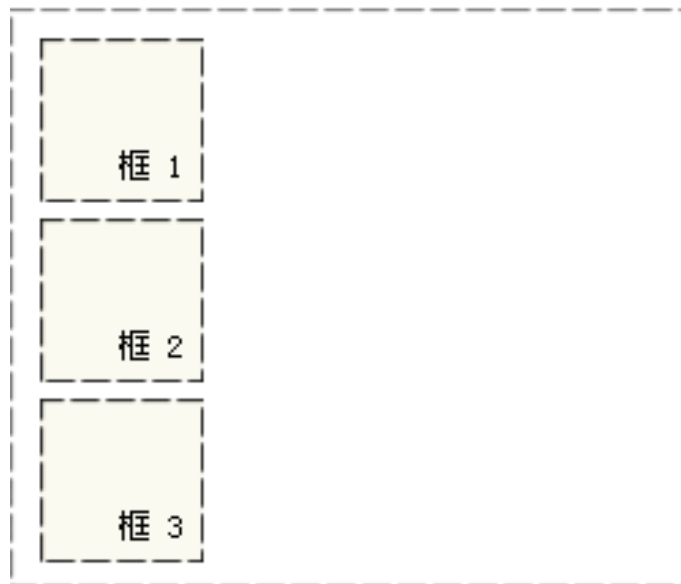
- 浮動的框可以向左或向右移動，直到它的外邊緣碰到包含框或另一個浮動框的邊框為止。
- 由於浮動框不在文檔的普通流中，所以文檔的普通流中的塊框表現得就像浮動框不存在一樣。



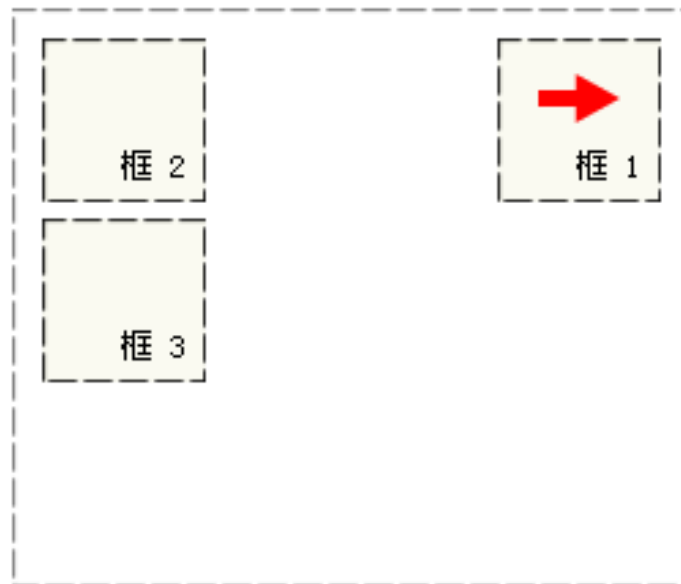
# CSS 定位

- 請看下圖，當把框1 向右浮動時，它脫離文檔流並且向右移動，直到它的右邊緣碰到包含框的右邊緣：

不浮動的框



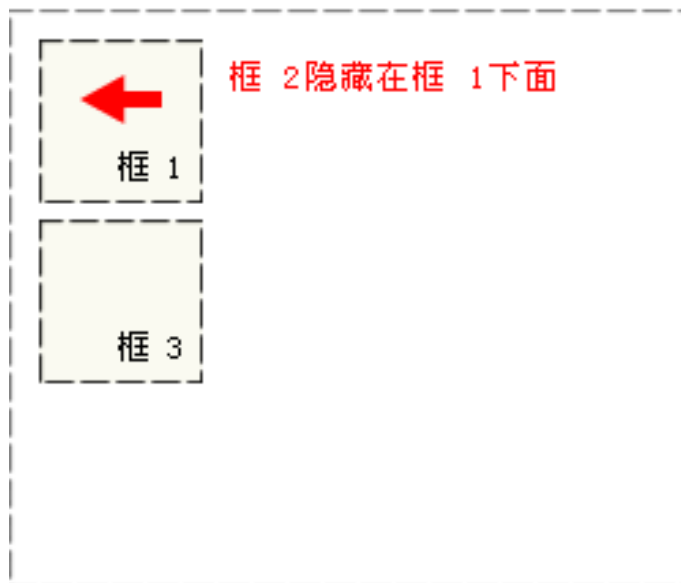
框 1 向右浮動



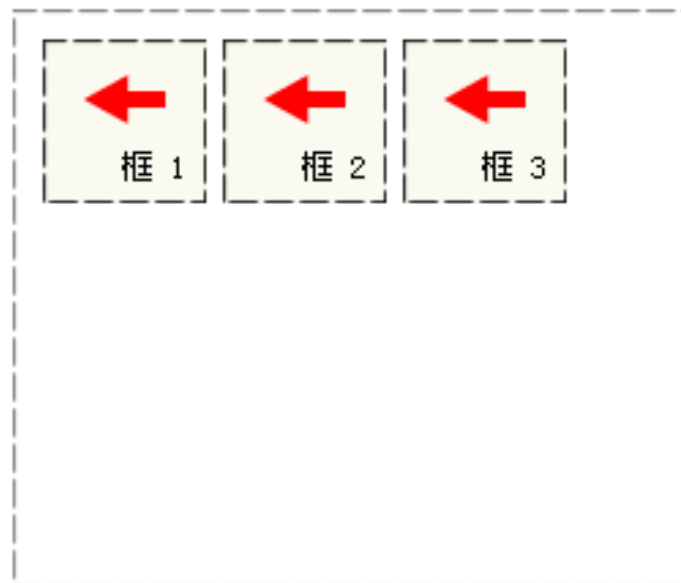
# CSS 定位

- 如果把所有三個框都向左移動，那麼框 1 向左浮動直到碰到包含框，另外兩個框向左浮動直到碰到前一個浮動框。

框 1 向左浮動



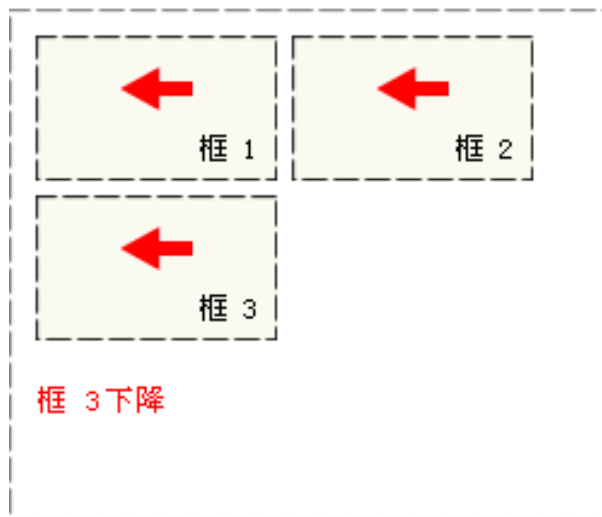
所有三個框向左浮動



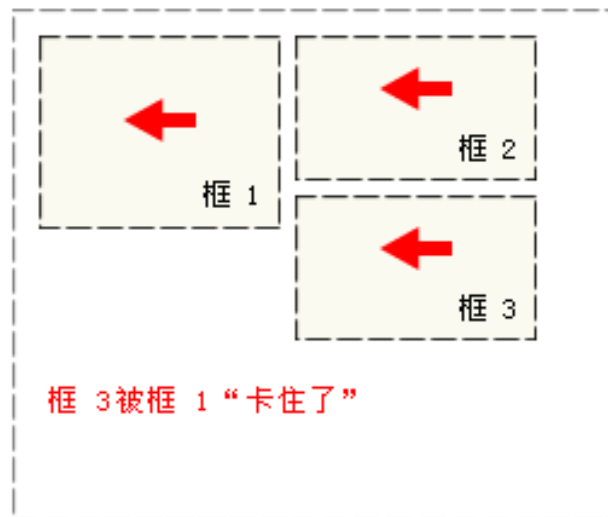
# CSS 定位

- 如下圖所示，如果外框太窄，無法容納水平排列的三個浮動元素，那麼其它浮動塊向下移動，直到有足夠的空間。如果浮動元素的高度不同，那麼當它們向下移動時可能被其它浮動元素「卡住」。

框 1 向左浮动



所有三个框向左浮动



# CSS 定位

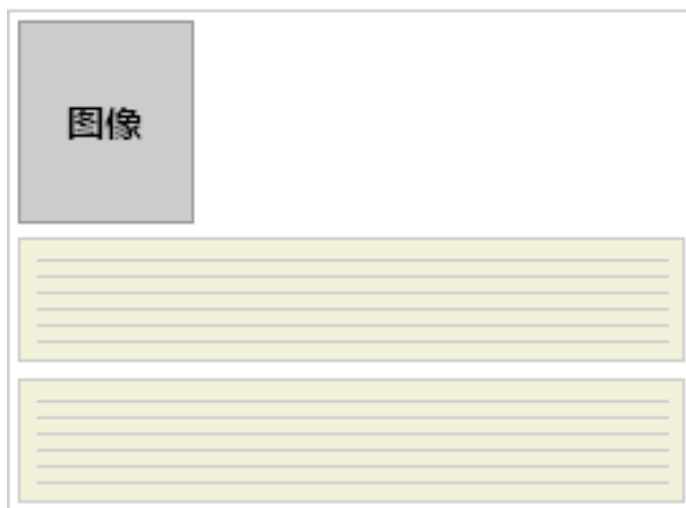
- CSS float 屬性
  - 在 CSS 中，我們通過 float 屬性實現元素的浮動。
  - 下例為把圖像向左浮動：

```
img { float: left }
```

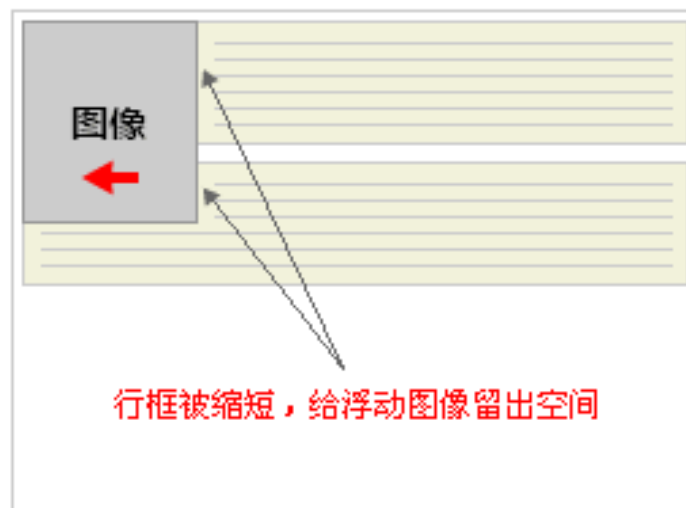
# CSS 定位

- 行框和清理
  - 浮動框旁邊的行框被縮短，從而給浮動框留出空間，行框圍繞浮動框。因此，創建浮動框可以使文本圍繞圖像：

不浮動的框



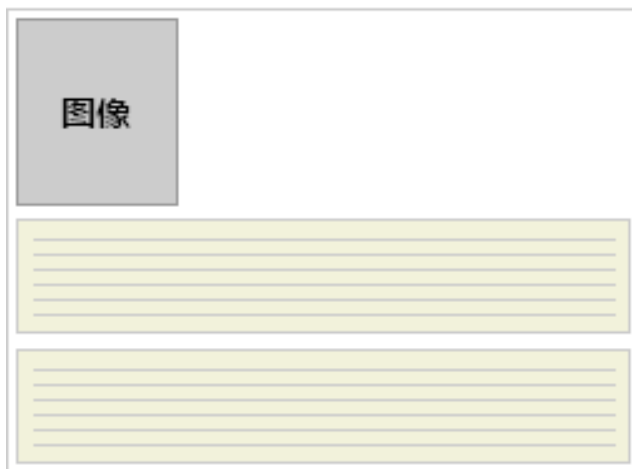
图像向左浮动



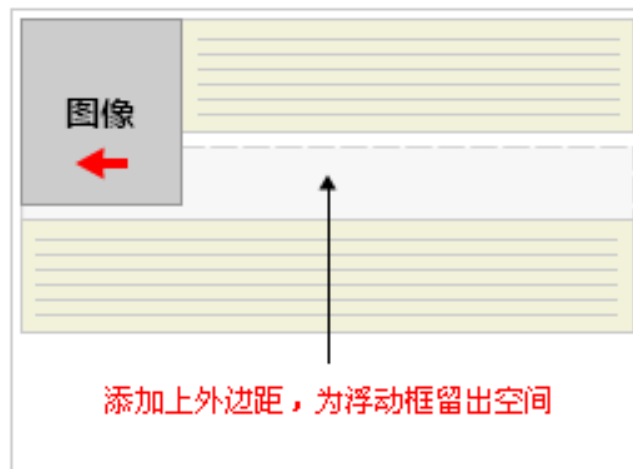
# CSS 定位

- 要想阻止行框圍繞浮動框，需要對該框應用 clear 屬性。Clear 屬性的值可以是 left、right、both 或 none，它表示框的哪些邊不應該挨著浮動框。
- 為了實現這種效果，在被清理的元素的上外邊距上添加足夠的空間，使元素的頂邊緣垂直下降到浮動框下面：

清理第二个段落



清理第二个段落

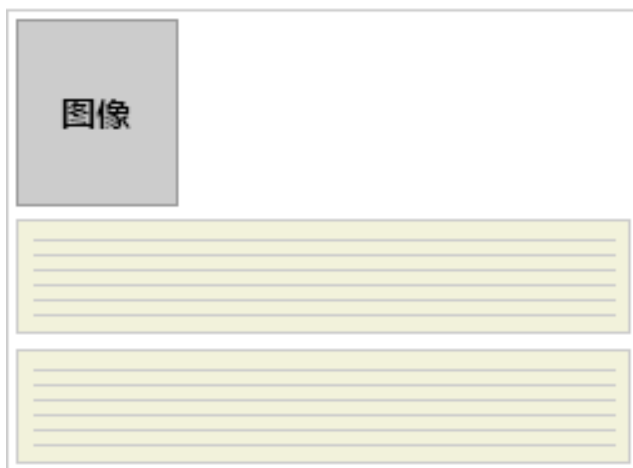


# CSS 定位

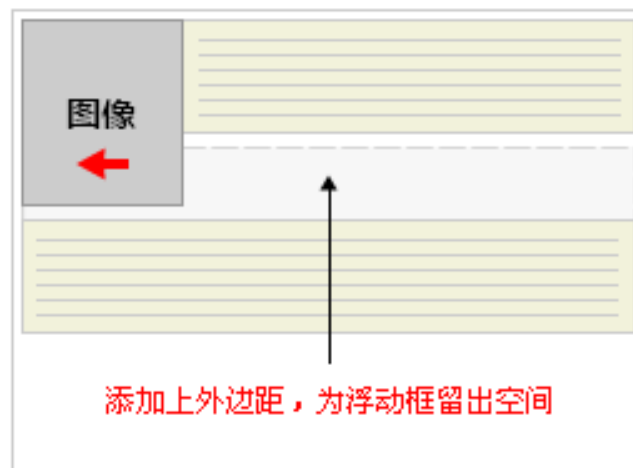
- 要想阻止行框圍繞浮動框，需要對該框應用 `clear` 屬性。  
`Clear` 屬性的值可以是 `left`、`right`、`both` 或 `none`，它表示框的哪些邊不應該挨著浮動框。
- 為了實現這種效果，在被清理的元素的上外邊距上添加足夠的空間，使元素的頂邊緣垂直下降到浮動框下面：

`img { clear: both }`

清理第二个段落



清理第二个段落



# 操作範例 web09

- 請完成下面頁面設計：

